

```
#!/usr/bin/env python3
"""
Quick validation script for skills - minimal version
"""

import sys
import re
import yaml
from pathlib import Path

# Directories whose contents are not packaged as part of the skill, so any
# SKILL.md inside them shouldn't count toward the single-SKILL.md check below.
# Mirrors package_skill.py: __pycache__ and node_modules are excluded at any
# depth, while evals is only excluded at the skill root.
EXCLUDED_DIR_PARTS = {'__pycache__', 'node_modules'}
ROOT_EXCLUDED_DIR_PARTS = {'evals'}

def _counts_as_skill_md(rel_path):
    """True if a SKILL.md at rel_path (relative to the skill root) would be
    packaged."""
    dir_parts = rel_path.parts[:-1]
    if any(part in EXCLUDED_DIR_PARTS for part in dir_parts):
        return False
    if dir_parts and dir_parts[0] in ROOT_EXCLUDED_DIR_PARTS:
        return False
    return True

def validate_skill(skill_path):
    """Basic validation of a skill"""
    skill_path = Path(skill_path)

    # Check SKILL.md exists
    skill_md = skill_path / 'SKILL.md'
    if not skill_md.exists():
        return False, "SKILL.md not found"

    # A skill must contain exactly one SKILL.md, at <folder>/SKILL.md. Extra
    # (nested) SKILL.md files are rejected on upload: the Skills API and claude.ai
    # accept exactly one per skill - only Claude Code's filesystem loads nested
    # ones. package_skill produces an upload-bound .skill, so block here rather
    # than ship an artifact that's guaranteed to fail on upload.
    skill_md_files = [
        p for p in skill_path.rglob('SKILL.md')
        if _counts_as_skill_md(p.relative_to(skill_path))
    ]
    if len(skill_md_files) > 1:
        extras = sorted(
            str(p.relative_to(skill_path)) for p in skill_md_files
            if p.resolve() != skill_md.resolve()
        )
        return False, (
            f"Found {len(skill_md_files)} SKILL.md files, but a skill must contain "
            f"exactly one at <folder>/SKILL.md. The Skills API and claude.ai reject "

```

```

        f"multiple on upload (only Claude Code's filesystem loads nested skills). "
        f"Extra: {'', ' '.join(extras)}.\n"
        "    - Separate skills: package each on its own, or build a plugin "
        f"(skills/<name>/SKILL.md).\n"
        "    - Supporting docs: rename to non-SKILL.md files (e.g. "
references/<topic>.md).\n"
        "    - Swept in by mistake: package only the one skill directory."
    )

# Read and validate frontmatter
content = skill_md.read_text()
if not content.startswith('---'):
    return False, "No YAML frontmatter found"

# Extract frontmatter
match = re.match(r'^---\n(?:.*?)\n---', content, re.DOTALL)
if not match:
    return False, "Invalid frontmatter format"

frontmatter_text = match.group(1)

# Parse YAML frontmatter
try:
    frontmatter = yaml.safe_load(frontmatter_text)
    if not isinstance(frontmatter, dict):
        return False, "Frontmatter must be a YAML dictionary"
except yaml.YAMLError as e:
    return False, f"Invalid YAML in frontmatter: {e}"

# Define allowed properties
ALLOWED_PROPERTIES = {'name', 'description', 'license', 'allowed-tools',
'metadata', 'compatibility'}

# Check for unexpected properties (excluding nested keys under metadata)
unexpected_keys = set(frontmatter.keys()) - ALLOWED_PROPERTIES
if unexpected_keys:
    return False, (
        f"Unexpected key(s) in SKILL.md frontmatter: {'', ' '.join(sorted(unexpected_keys))}. "
        f"Allowed properties are: {'', ' '.join(sorted(ALLOWED_PROPERTIES))}"
    )

# Check required fields
if 'name' not in frontmatter:
    return False, "Missing 'name' in frontmatter"
if 'description' not in frontmatter:
    return False, "Missing 'description' in frontmatter"

# Extract name for validation
name = frontmatter.get('name', '')
if not isinstance(name, str):
    return False, f"Name must be a string, got {type(name).__name__}"
name = name.strip()
if name:
    # Check naming convention (kebab-case: lowercase with hyphens)

```

```

        if not re.match(r'^[a-z0-9-]+$ ', name):
            return False, f"Name '{name}' should be kebab-case (lowercase letters,
digits, and hyphens only)"
        if name.startswith('-') or name.endswith('-') or '--' in name:
            return False, f"Name '{name}' cannot start/end with hyphen or contain
consecutive hyphens"
        # Check name length (max 64 characters per spec)
        if len(name) > 64:
            return False, f"Name is too long ({len(name)} characters). Maximum is 64
characters."

    # Extract and validate description
    description = frontmatter.get('description', '')
    if not isinstance(description, str):
        return False, f"Description must be a string, got {type(description).__name__}"
    description = description.strip()
    if description:
        # Check for angle brackets
        if '<' in description or '>' in description:
            return False, "Description cannot contain angle brackets (< or >)"
        # Check description length (max 1024 characters per spec)
        if len(description) > 1024:
            return False, f"Description is too long ({len(description)} characters).
Maximum is 1024 characters."

    # Validate compatibility field if present (optional)
    compatibility = frontmatter.get('compatibility', '')
    if compatibility:
        if not isinstance(compatibility, str):
            return False, f"Compatibility must be a string, got
{type(compatibility).__name__}"
        if len(compatibility) > 500:
            return False, f"Compatibility is too long ({len(compatibility)}
characters). Maximum is 500 characters."

    return True, "Skill is valid!"

if __name__ == "__main__":
    if len(sys.argv) != 2:
        print("Usage: python quick_validate.py <skill_directory>")
        sys.exit(1)

    valid, message = validate_skill(sys.argv[1])
    print(message)
    sys.exit(0 if valid else 1)

```